Unit -3

Bernoulli Naive Bayes

- **Bernoulli Naive Bayes (BNB) Classification** is a variant of Naive Bayes designed for **binary/Boolean features**.
- It assumes that features are binary **(0 or 1) represents presence (1) or absence (0)** of specific **words** and is commonly used for **text classification** tasks such as **spam detection** or **sentiment analysis**.

Example of Naïve Bayes Classifier

Name	Give Birth	Can Fly	Live in Water	Have Legs	Class
human	yes	no	no	yes	mammals
python	no	no	no	no	non-mammals
salmon	no	no	yes	no	non-mammals
whale	yes	no	yes	no	mammals
frog	no	no	sometimes	yes	non-mammals
komodo	no	no	no	yes	non-mammals
bat	yes	yes	no	yes	mammals
pigeon	no	yes	no	yes	non-mammals
cat	yes	no	no	yes	mammals
leopard shark	yes	no	yes	no	non-mammals
turtle	no	no	sometimes	yes	non-mammals
penguin	no	no	sometimes	yes	non-mammals
porcupine	yes	no	no	yes	mammals
eel	no	no	yes	no	non-mammals
salamander	no	no	sometimes	yes	non-mammals
gila monster	no	no	no	yes	non-mammals
platypus	no	no	no	yes	mammals
owl	no	yes	no	yes	non-mammals
dolphin	yes	no	yes	no	mammals
eagle	no	yes	no	yes	non-mammals

A: attributes

M: mammals

N: non-mammals

$$P(A \mid M) = \frac{6}{7} \times \frac{6}{7} \times \frac{2}{7} \times \frac{2}{7} = 0.06$$

$$P(A \mid N) = \frac{1}{13} \times \frac{10}{13} \times \frac{3}{13} \times \frac{4}{13} = 0.0042$$

$$P(A \mid M)P(M) = 0.06 \times \frac{7}{20} = 0.021$$

$$P(A \mid N)P(N) = 0.004 \times \frac{13}{20} = 0.0027$$

Give Birth	Can Fly	Live in Water	Have Legs	Class
yes	no	yes	no	?

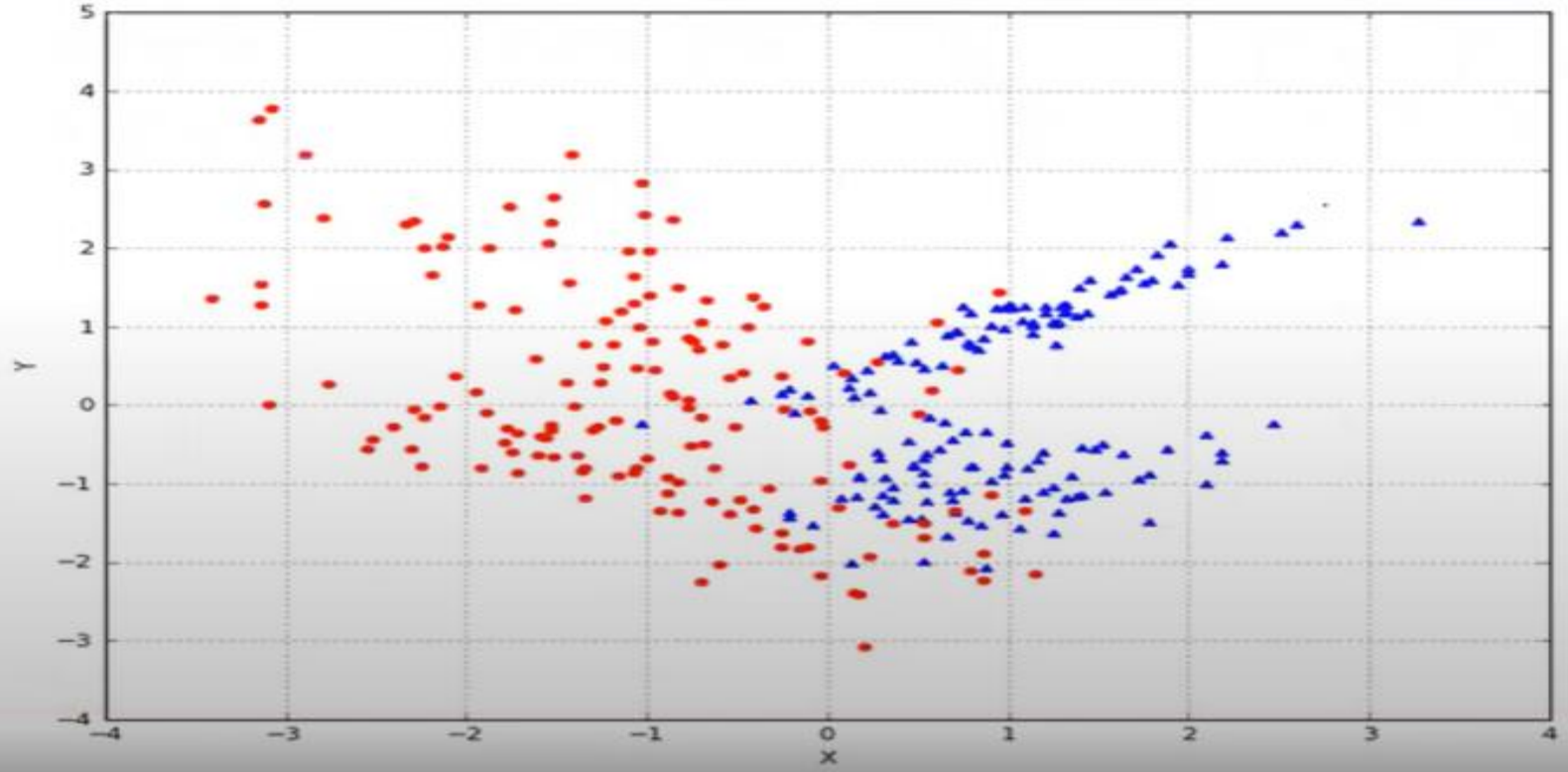
P(A|M)P(M) > P(A|N)P(N)

=> Mammals

Bernoulli naive Bayes

SK-Learn: to create dummy dataset (Bi dimensional)

```
from sklearn.datasets import make_classification
>>> nb_samples = 300
>>> X, Y = make_classification (n_samples=nb_samples ,
n_features=2, n_informative=2, n_redundant=0)
```



SK-Learn Bernoulli NB

```
from sklearn.naive_bayes import BernoulliNB
from sklearn.model_selection import train_test_split

X_train, X_test, Y_train, Y_test = train_test_split(X, Y,test_size=0.25)

bnb = BernoulliNB(binarize=0.0)

bnb.fit(X_train, Y_train)

bnb.score(X_test, Y_test)
0.8533333333333339
```

.....

Example: Spam Email Classification:

Dataset:

We will use a **small binary dataset** with **features** representing the **presence (1) or absence (0)** of specific **words** in an **email**.

Email ID	Word: "Offer"	Word: "Win"	Word: "Click"	Spam (Label)
1	1	1	1	1 (Spam)
2	1	0	1	1 (Spam)
3	0	0	0	0 (Non-Spam)
4	0	1	1	0 (Non-Spam)
5	1	1	0	1 (Spam)

Sample SMS Dataset

File Edit View

```
ham     Go until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...
ham     Ok lar... Joking wif u oni...
spam    Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text FA to 87121 to receive entry question(std txt rate)T&C's apply 08452810075over18's
ham     U dun say so early hor... U c already then say...
ham     Nah I don't think he goes to usf, he lives around here though
spam    FreeMsg Hey there darling it's been 3 week's now and no word back! I'd like some fun you up for it still? Tb ok! XxX std chgs to send, £1.50 to rcv
ham     Even my brother is not like to speak with me. They treat me like aids patent.
ham     As per your request 'Melle Melle (Oru Minnaminunginte Nurungu Vettam)' has been set as your callertune for all Callers. Press *9 to copy your friends Callertune
spam    WINNER!! As a valued network customer you have been selected to receivea £900 prize reward! To claim call 09061701461. Claim code KL341. Valid 12 hours only.
spam    Had your mobile 11 months or more? U R entitled to Update to the latest colour mobiles with camera for Free! Call The Mobile Update Co FREE on 08002986030
ham     I'm gonna be home soon and i don't want to talk about this stuff anymore tonight, k? I've cried enough today.
spam    SIX chances to win CASH! From 100 to 20,000 pounds txt> CSH11 and send to 87575. Cost 150p/day, 6days, 16+ TsandCs apply Reply HL 4 info
spam    URGENT! You have won a 1 week FREE membership in our £100,000 Prize Jackpot! Txt the word: CLAIM to No: 81010 T&C www.dbuk.net LCCLTD POBOX 4403LDNW1A7RW18
ham     I've been searching for the right words to thank you for this breather. I promise i wont take your help for granted and will fulfil my promise. You have been wonderful and a
blessing at all times.
ham     I HAVE A DATE ON SUNDAY WITH WILL!!
spam    XXXMobileMovieClub: To use your credit, click the WAP link in the next txt message or click here>> http://wap. xxxmobilemovieclub.com?n=QJKGIGHJJGCBL
ham     Oh k...i'm watching here:)
ham     Eh u remember how 2 spell his name... Yes i did. He v naughty make until i v wet.
ham     Fine if that⍰s the way u feel. That⍰s the way its gota b
spam    England v Macedonia - dont miss the goals/team news. Txt ur national team to 87077 eg ENGLAND to 87077 Try:WALES, SCOTLAND 4txt/ú1.20 POBOXox36504W45WQ 16+
ham     Is that seriously how you spell his name?
ham     I‘m going to try for 2 months ha ha only joking
ham     So ü pay first lar... Then when is da stock comin...
ham     Aft i finish my lunch then i go str down lor. Ard 3 smth lor. U finish ur lunch already?
ham     Ffffffffff. Alright no way I can meet up with you sooner?
ham     Just forced myself to eat a slice. I'm really not hungry tho. This sucks. Mark is getting worried. He knows I'm sick when I turn down pizza. Lol
ham     Lol your always so convincing.
ham     Did you catch the bus ? Are you frying an egg ? Did you make a tea? Are you eating your mom's left over dinner ? Do you feel my Love ?
ham     I'm back &amp;amp; we're packing the car now, I'll let you know if there's room
ham     Ahhh. Work. I vaguely remember that! What does it feel like? Lol
ham     Wait that's still not all that clear, were you not sure about me being sarcastic or that that's why x doesn't want to live with us
ham     Yeah he got in at 2 and was v apologetic. n had fallen out and she was actin like spoilt child and he got caught up in that. Till 2! But we won't go there! Not doing too badly
cheers. You?
ham     K tell me anything about you.
ham     For fear of fainting with the of all that housework you just did? Quick have a cuppa
spam    Thanks for your subscription to Ringtone UK your mobile will be charged £5/month Please confirm by replying YES or NO. If you reply NO you will not be charged
ham     Yup... Ok i go home look at the timings then i msg ü again... Xuhui going to learn on 2nd may too but her lesson is at 8am
ham     Oops, I'll let you know when my roommate's done
ham     I see the letter B on my car
```

Step 2: Formula for Bernoulli Naive Bayes

For a given **email x** (features), we calculate the probability of each **class (C)** using **Bayes' theorem**:

$$P(C|\mathbf{x}) = \frac{\boxed{P(C) \cdot P(\mathbf{x}|C)}}{P(\mathbf{x})}$$

Since $P(x)$ is constant across all **classes**, we only calculate the numerator

P(C): Prior probability of the **class (C).**

P(x|C): Likelihood of feature x given **class** C

Step 3: Train the Model

Calculate Prior Probability $P(C)$:

$$P(\text{Spam}) = \frac{\text{Number of Spam Emails}}{\text{Total Emails}} = \frac{3}{5} = 0.6$$

$$P(\text{Non-Spam}) = \frac{\text{Number of Non-Spam Emails}}{\text{Total Emails}} = \frac{2}{5} = 0.4$$

……..

Calculate Likelihoods $P(xi|C)$:

For each **word (feature),** compute the probability of the word's **presence (xi=1)** or **absence (xi=0)** in **emails** of **each class**.

Use **Laplace Smoothing** to avoid **zero probabilities**:

Laplace smoothing is used to handle **zero probabilities** in **probabilistic models**, especially in **Naive Bayes.**

It adds a small constant **(usually 1)** to all counts to ensure **no probability becomes zero**, which could otherwise **invalidate predictions**.

.....

The formula for a conditional probability with **Laplace smoothing** is:

$$P(x_i = 1|C) = \frac{(\text{Count of } x_i = 1 \text{ in class } C) + \alpha}{(\text{Total instances in class } C) + \alpha \cdot k}$$

Where:

xi : A feature (e.g., **presence/absence** of a word).

C: A class (e.g., **Spam or Non-Spam**).

α: Smoothing parameter (**typically *α*=1**).

k: Number of unique **feature** in the dataset (***k*=3**)

……

- Let's calculate the likelihood $P(xi=1|C)$ $P(xi=0|C)$ for each feature in the **Spam and Non-Spam** classes using **Laplace Smoothing** from the example.
- **Using Laplace Smoothing (α=1,k=2)**:

1.Offer (Spam)

$$\boxed{P(x_i = 1|\text{Spam})} = \frac{\text{Count of "}\boxed{\text{Offer}}\text{" in Spam} + \alpha}{n_{\text{spam}} + \alpha \cdot k} = \frac{3+1}{3+2} = \frac{4}{5} = \boxed{0.8}$$

$$\boxed{P(x_i = 0|\text{Spam})} = 1 - P(x_i = 1|\text{Spam}) = 1 - 0.8 = \boxed{0.2}$$

.....

2. Win (Spam)

$$\boxed{P(x_i = 1|\text{Spam})} = \frac{\text{Count of } \boxed{\text{"Win"}} \text{ in Spam} + \alpha}{n_{\text{spam}} + \alpha \cdot k} = \frac{2+1}{3+2} = \frac{3}{5} = \boxed{0.6}$$

$$\boxed{P(x_i = 0|\text{Spam})} = 1 - P(x_i = 1|\text{Spam}) = 1 - 0.6 = \boxed{0.4}$$

.....

3.Click (Spam)

$$\boxed{P(x_i = 1|\text{Spam})} = \frac{\text{Count of "}\boxed{\text{Click}}\text{" in Spam} + \alpha}{n_{\text{spam}} + \alpha \cdot k} = \frac{2+1}{3+2} = \frac{3}{5} = \boxed{0.6}$$

$$\boxed{P(x_i = 0|\text{Spam})} = 1 - P(x_i = 1|\text{Spam}) = 1 - 0.6 = \boxed{0.4}$$

.....

4.Offer(Non-Spam)

$$\boxed{P(x_i = 1|\text{Non-Spam})} = \frac{\text{Count of "}\boxed{\text{Offer}}\text{" in Non-Spam} + \alpha}{n_{\text{non-spam}} + \alpha \cdot k} = \frac{0+1}{2+2} = \frac{1}{4} = \boxed{0.25}$$

$$\boxed{P(x_i = 0|\text{Non-Spam})} = 1 - P(x_i = 1|\text{Non-Spam}) = 1 - 0.25 = \boxed{0.75}$$

....

5. Win (Non-Spam)

$$\boxed{P(x_i = 1|\text{Non-Spam})} = \frac{\text{Count of "}\boxed{\text{Win}}\text{" in Non-Spam} + \alpha}{n_{\text{non-spam}} + \alpha \cdot k} = \frac{1+1}{2+2} = \frac{2}{4} = \boxed{0.5}$$

$$\boxed{P(x_i = 0|\text{Non-Spam})} = 1 - P(x_i = 1|\text{Non-Spam}) = 1 - 0.5 = \boxed{0.5}$$

.....

6.Click(Non-Spam)

$$\boxed{P(x_i = 1|\text{Non-Spam})} = \frac{\text{Count of "}\boxed{\text{Click}}\text{" in Non-Spam} + \alpha}{n_{\text{non-spam}} + \alpha \cdot k} = \frac{2+1}{2+2} = \frac{3}{4} = \boxed{0.75}$$

$$\boxed{P(x_i = 0|\text{Non-Spam})} = 1 - P(x_i = 1|\text{Non-Spam}) = 1 - 0.75 = \boxed{0.25}$$

Step 4: Predict a New Email

Given Email:

Features: {"Offer": 1, "Win": 1, "Click": 0}

Compute P(Spam|x) and P(Non-Spam|x).

Spam Probability : P(Spam|x):

P(Spam|x) = P(spam) . P(x|Spam) // Bayes theorem

P(spam) =0.6

P(x|spam)=P(offer=1|spam).P(Win=1|spam).P(click=0|spam)

= 0.8.0.6.0.4 = 0.192

P(Spam|x) = 0.6 x 0.192 = 0.1152

.....

Non Spam Probability : P(Non Spam|x):

P(Non Spam|x) = P(Non spam) . P(x|Non Spam) // Bayes theorem

P(Non spam) = 0.4

P(x|Non spam)=P(offer=1|Non spam).P(Win=1|Non spam).P(click=0|Non spam)

= 0.25.0.5.0.25 = 0.03125

P(Non Spam|x) = 0.4 x 0.03125 = 0.0125

Step 5: Compare and Predict

P(Spam|x)=0.1152, P(Non-Spam|x)=0.0125

Since **P(Spam|x)**>**P(Non-Spam|x)** , the model predicts the email as **Spam**.

bernaulis1.ipynb

File Edit View Insert Runtime Tools Help All changes saved

+ Code + Text

6s

```python
from sklearn.naive_bayes import BernoulliNB
import numpy as np
```

+ Code + Text

0s

```python
# Dataset
X = np.array([
    [1, 1, 0],  # Email 1 features: "Free", "Win", "Cash"
    [0, 1, 1],  # Email 2 features
    [1, 0, 0],  # Email 3 features
    [0, 0, 1]   # Email 4 features
])
y = np.array([1, 1, 0, 0])  # Labels: 1 for Spam, 0 for Not Spam
```

```python
# New email to classify
new_email = np.array([[1, 1, 0]])  # "Free" = 1, "Win" = 1, "Cash" = 0
```

```
[4] # Initialize and fit the model
    model = BernoulliNB()
    model.fit(X, y)
```

```
BernoulliNB
BernoulliNB()
```

```
[5] prediction = model.predict(new_email)
```

```
print(prediction)
```

```
[1]
```

```
probablities = model.predict_proba(new_email)
print(probablities)
```

```
[[0.25 0.75]]
```

probabilities is a 2D array each **column** gives the probability of a specific class **(column 0 = Not Spam, column 1 = Spam)**

Multinomial Naive Bayes (MNB)

The **Multinomial Naive Bayes (MNB)** model is commonly used for **text classification** tasks, such as **spam detection**, where **features** represent **frequencies or counts of specific words**(**e.g., word frequencies in emails).**

Multinomial Naive Bayes: Key Points

- The term "Naive" means the algorithm assumes all features (such as words in a sentence) are independent of each other, even though this may not be true in real text data.
- "Multinomial" refers to considering the frequency or count of each feature, such as how many times a word appears in a document or category.
- It classifies text by using word counts, not just the presence or absence of words.
- The core idea is that the probability of each word appearing is calculated independently for each category.
- The model checks how often each word appears in each class (for example, "spam" vs. "not spam") and uses these counts to make predictions.
- For instance, if the word "free" appears frequently in spam emails, the model learns this is a strong indicator of spam.
- This independence assumption makes the method simple and efficient for text classification tasks like spam detection or sentiment analysis.

To calculate the probability of a message belonging to a certain category Multinomial Naive Bayes uses the **multinomial distribution**:

$$P(X) = \frac{n!}{n_1!n_2!...n_m!} p_1^{n_1} p_2^{n_2} \cdots p_m^{n_m}$$

Where:

- n is the total number of trials.
- n_i is the count of occurrences for outcome i.
- p_i is the probability of outcome i.

To estimate how likely each word is in a particular class like "spam" or "not spam" we use a method called **Maximum Likelihood Estimation (MLE)**. This helps finding probabilities based on actual counts from our data. The formula is:

$$\theta_{c,i} = \frac{\text{count}(w_i, c)+1}{N+v}$$

Where:

- $count(wi, c)$ is the number of times word w_i appears in documents of class c.
- N is the total number of words in documents of class c*c*.
- v is the vocabulary size.

Example

To understand how Multinomial Naive Bayes works, here's a simple example to classify whether a message is **"spam"** or **"not spam"** based on the presence of certain words.

Message ID	Message Text	Class
M1	"buy cheap now"	Spam
M2	"limited offer buy"	Spam
M3	"meet me now"	Not Spam
M4	"let's catch up"	Not Spam

1. Vocabulary

Extract all unique words from the training data:

$$\text{Vocabulary} = \{\text{buy, cheap, now, limited, offer, meet, me, let's, catch, up}\}$$

Vocabulary size $V = 10$

2. Word Frequencies by Class

Spam Class (M1, M2):

- buy: 2
- cheap: 1
- now: 1
- limited: 1
- offer: 1

Total words: 6

Not Spam Class (M3, M4):

- meet: 1
- me: 1
- now: 1
- let's: 1
- catch: 1
- up: 1

Total words: 6

3. Test Message

Test Message: "buy now"

4. Applying Multinomial Naive Bayes Formula

$$P(C|d) \propto P(C) \cdot \prod_i P(w_i|C)^{f_i}$$

Prior Probabilities:

$$P(\text{Spam}) = 0.5, \quad P(\text{Not Spam}) = 0.5$$

Apply Laplace Smoothing:

$$P(w \mid C) = \frac{\text{count}(w,C)+1}{\text{total words in } C+V}$$

Spam Class:

- $P(\text{buy} \mid \text{Spam}) = \frac{2+1}{6+10} = \frac{3}{16}$
- $P(\text{now} \mid \text{Spam}) = \frac{1+1}{6+10} = \frac{2}{16}$

$$P(\text{Spam} \mid d) \propto 0.5 \cdot \frac{3}{16} \cdot \frac{2}{16} = \frac{3}{256}$$

Not Spam Class:

- $P(\text{buy} \mid \text{Not Spam}) = \frac{0+1}{6+10} = \frac{1}{16}$
- *P(\text{now} \mid \text{Not Spam}) = \frac{1 + 1}{6 + 10} = \frac{2}{16}*

$$P(\text{Not Spam} \mid d) \propto 0.5 \cdot \frac{1}{16} \cdot \frac{2}{16} = \frac{1}{256}$$

5. Final Classification

Since, $P(\text{Spam}|d) = \frac{3}{256} > \frac{1}{256} = P(\text{Not Spam}|d)$

$\boxed{\text{The message is classified as Spam}}$

Python Implementation of Multinomial Naive Bayes

Let's understand it with a example of spam email detection. We'll classify emails into two categories: **spam** and **not spam**.

1. Importing Libraries:

We will import pandas and scikit learn where:

- **pandas**: Used for handling data in DataFrame format.
- **CountVectorizer**: Converts a collection of text documents into a matrix of token counts.
- **train_test_split**: Splits the data into training and test sets for model evaluation.
- **MultinomialNB**: A Naive Bayes classifier suited for classification tasks with discrete features (such as word counts).
- **accuracy_score**: Computes the accuracy of the model's predictions.

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score
```

2. Creating the Dataset

A simple dataset is created with text messages labeled as either spam or not spam. This data is then converted into a DataFrame for easy handling.

```
data = {
    'text': [
        'Free money now',
        'Call now to claim your prize',
        'Meet me at the park',
        'Let’s catch up later',
        'Win a new car today!',
        'Lunch plans?',
        'Congratulations! You won a lottery',
        'Can you send me the report?',
        'Exclusive offer for you',
        'Are you coming to the meeting?'
    ],
    'label': ['spam', 'spam', 'not spam', 'not spam', 'spam', 'not spam', 'spam', 'not spam', 'spam', 'not
spam']
}

df = pd.DataFrame(data)
```

3. Mapping Labels to Numerical Values

The labels (spam and not spam) are mapped to numerical values where spam becomes 1 and not spam becomes 0. This is necessary for the classifier, as it works with numerical data.

```
df['label'] = df['label'].map({'spam': 1, 'not spam': 0})
```

4. Splitting the Data

- X contains the text messages (features), and y contains the labels (target).
- The dataset is split into training (70%) and testing (30%) sets using **train_test_split**.

```
X = df['text']
y = df['label']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

5. Vectorizing the Text Data

- **CountVectorizer** is used to convert text data into numerical vectors. It counts the occurrences of each word in the corpus.
- **fit_transform()** is applied to the training data to learn the vocabulary and transform it into a feature matrix.
- **transform()** is applied to the test data to convert it into the same feature space.

```
vectorizer = CountVectorizer()
X_train_vectors = vectorizer.fit_transform(X_train)
X_test_vectors = vectorizer.transform(X_test)
```

6. Training the Naive Bayes Model

A Multinomial Naive Bayes classifier is created and trained using the vectorized training data (**X_train_vectors**) and corresponding labels (**y_train**).

```
model = MultinomialNB()
model.fit(X_train_vectors, y_train)
```

7. Making Predictions and Evaluating Accuracy

- We are using **model.predict(X_test_vectors)** to generate predictions from the trained model on test data.
- **accuracy_score(y_test, y_pred)** compares predicted labels **y_pred** with true labels **y_test** to calculate accuracy.

```
y_pred = model.predict(X_test_vectors)
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy * 100:.2f}%\n")
```

Output:

Accuracy: 66.67%

8. Predicting for a Custom Message

- We create a custom message and transform it into a vector using **vectorizer.transform().**
- The vectorized message is passed to **model.predict()** to get the prediction.
- We print the result, interpreting 1 as “Spam” and 0 as “Not Spam”.

```
custom_message = ["Congratulations, you've won a free vacation"]
print(custom_message)
custom_vector = vectorizer.transform(custom_message)
prediction = model.predict(custom_vector)
print("Prediction for custom message:", "Spam" if prediction[0] == 1 else "Not Spam")
```

Output:

Congratulations, you've won a free vacation
Prediction for custom message: Spam

In the above code we did spam detection for given set of messages and evaluated model accuracy for the output it gave.

Example 2:
Step 1: Problem Setup

Classify emails as **Spam** or **Non-Spam** using **word counts**.

Example Dataset:

Email ID	Word: "Offer"	Word: "Win"	Word: "Click"	Word: "You"	Spam (Label)
1	2	3	1	0	1 (Spam)
2	1	0	2	1	1 (Spam)
3	0	0	0	3	0 (Non-Spam)
4	1	1	1	2	0 (Non-Spam)

Step 2: Formula for Multinomial Naive Bayes (MNB)

For a given **email x** (features), we calculate the probability of each **class (C)** using **Bayes' theorem**:

$$P(C|\mathbf{x}) = \frac{\boxed{P(C) \cdot P(\mathbf{x}|C)}}{P(\mathbf{x})}$$

Since $P(x)$ is constant across all **classes**, we only calculate the numerator

P(C): Prior probability of the **class (C).**

P(x|C): Likelihood of feature x given **class** C

Step 3: Train the Model

A. Calculate Priors ($P(C)$)

Total emails = 4

P(Spam) = $\frac{4}{2}$ =0.5

P(Non-Spam) = $\frac{4}{2}$ =0.5

B.Calculate total Word Counts in Each Class

1. **For Spam Class:**

Emails labeled as Spam: **Email 1** and **Email 2**

Word counts for each email:

- Email 1: $[2, 3, 1, 0]$
- Email 2: $[1, 0, 2, 1]$

Total word counts for Spam:

$$\text{Total word count (Spam)} = (2+1) + (3+0) + (1+2) + (0+1) = \boxed{10}$$

.....

2. **For Non-Spam Class:**

Emails labeled as Non-Spam: **Email 3** and **Email 4**

Word counts for each email:

- Email 3: $[0, 0, 0, 3]$
- Email 4: $[1, 1, 1, 2]$

Total word counts for Non-Spam:

$$\text{Total word count (Non-Spam)} = (0+1) + (0+1) + (0+1) + (3+2) = \boxed{9}$$

C.Summarize Total Word Counts

Class	Total Word Count
Spam	10
Non-Spam	9

D. Individual Word Counts per Class

For each word in the vocabulary **({ "Offer", "Win", "Click", "You" })**, **sum** the **counts** in **each class**:

Example Dataset:

Email ID	Word: "Offer"	Word: "Win"	Word: "Click"	Word: "You"	Spam (Label)
1	2	3	1	0	1 (Spam)
2	1	0	2	1	1 (Spam)
3	0	0	0	3	0 (Non-Spam)
4	1	1	1	2	0 (Non-Spam)

1. **For Spam Class:**

$$\text{"Offer"} = 2 + 1 = 3, \quad \text{"Win"} = 3 + 0 = 3$$

$$\text{"Click"} = 1 + 2 = 3, \quad \text{"You"} = 0 + 1 = 1$$

2. **For Non-Spam Class:**

$$\text{"Offer"} = 0 + 1 = 1, \quad \text{"Win"} = 0 + 1 = 1$$

$$\text{"Click"} = 0 + 1 = 1, \quad \text{"You"} = 3 + 2 = 5$$

E. Calculate Likelihoods ($P(xi|C)$) Using Laplace Smoothing:

Using the formula for **Multinomial Naive Bayes** likelihood with **Laplace smoothing** (α=1, k=4):

$$P(x_i|C) = \frac{\text{Count of word } i \text{ in class } C + \alpha}{\text{Total word count in class } C + \alpha \cdot k}$$

Likelihoods for Spam Class:

- $P(\text{"Offer"}|\text{Spam}) = \frac{3+1}{10+4} = \frac{4}{14} = \boxed{0.286}$
- $P(\text{"Win"}|\text{Spam}) = \frac{3+1}{10+4} = \frac{4}{14} = \boxed{0.286}$
- $P(\text{"Click"}|\text{Spam}) = \frac{3+1}{10+4} = \frac{4}{14} = \boxed{0.286}$
- $P(\text{"You"}|\text{Spam}) = \frac{1+1}{10+4} = \frac{2}{14} = \boxed{0.143}$

.....

Likelihoods for Non-Spam Class:

- $P(\text{"Offer"}|\text{Non-Spam}) = \frac{1+1}{9+4} = \frac{2}{13} = \boxed{0.154}$
- $P(\text{"Win"}|\text{Non-Spam}) = \frac{1+1}{9+4} = \frac{2}{13} = \boxed{0.154}$
- $P(\text{"Click"}|\text{Non-Spam}) = \frac{1+1}{9+4} = \frac{2}{13} = \boxed{0.154}$
- $P(\text{"You"}|\text{Non-Spam}) = \frac{5+1}{9+4} = \frac{6}{13} = \boxed{0.462}$

P(x i |Spam)				P(x i |Non Spam)			
Offer	Win	Click	You	Offer	Win	Click	You
0.286	0.286	0.286	0.143	0.154	0.154	0.154	0.462

Step 4: New Email

We'll classify this email as either **Spam** or **Non-Spam** by calculating the posterior probabilities for each **class** using the formula:

Word	Count (x_i)
"Offer"	1
"Win"	1
"Click"	0
"You"	2

$$P(C|X) = P(X|C).P(C)$$

The likelihood formula:

$$P(\mathbf{x}|C) = \prod_{i=1}^{n} P(x_i|C)^{x_i}$$

where:

- $P(x_i|C)$: Probability of the i-th word given class C.
- x_i: Frequency of the i-th word in the document.

Step 5: Calculate the posterior probability

Calculate Posterior for Spam:

1. **Prior:**

$$\boxed{P(\text{Spam}) = 0.5}$$

2. **Likelihood:**

$$P(\mathbf{x}|\text{Spam}) = P(\text{"Offer"}|\text{Spam})^1 \cdot P(\text{"Win"}|\text{Spam})^1 \cdot P(\text{"Click"}|\text{Spam})^0 \cdot P(\text{"You"}|\text{Spam})^2$$

Substitute values:

$$P(\mathbf{x}|\text{Spam}) = \boxed{(0.286)^1} \cdot \boxed{(0.286)^1} \cdot \boxed{(0.286)^0} \cdot \boxed{(0.143)^2}$$

Compute:

$$P(\mathbf{x}|\text{Spam}) = 0.286 \cdot 0.286 \cdot 1 \cdot (0.143)^2$$

$$P(\mathbf{x}|\text{Spam}) = 0.286 \cdot 0.286 \cdot 0.020449 = \boxed{0.001667}$$

Posterior for Spam = P(Spam | X) = 0.5 x 0.001667 =0.000833

....

Calculate Posterior for Non-Spam

1. **Prior:**

$$P(\text{Non-Spam}) = \boxed{0.5}$$

2. **Likelihood:**

$$P(\mathbf{x}|\text{Non-Spam}) = P(\text{"Offer"}|\text{Non-Spam})^1 \cdot P(\text{"Win"}|\text{Non-Spam})^1 \cdot P(\text{"Click"}|\text{Non-Spam})^0 \cdot P(\text{"You"}|\text{Non-Spam})^2$$

Substitute values:

$$P(\mathbf{x}|\text{Non-Spam}) = \boxed{(0.154)^1} \cdot \boxed{(0.154)^1} \cdot \boxed{(0.154)^0} \cdot \boxed{(0.462)^2}$$

Compute:

$$P(\mathbf{x}|\text{Non-Spam}) = 0.154 \cdot 0.154 \cdot 1 \cdot (0.462)^2$$

$$P(\mathbf{x}|\text{Non-Spam}) = 0.154 \cdot 0.154 \cdot 0.213444 = \boxed{0.005057}$$

Posterior for Non-Spam = P(Non-Spam|X) = 0.5 x 0.005057 =0.002528

Step 6: Classification

The **new email** is classified as **Non-Spam** because:

$P(\textbf{Non-Spam}|x) > P(\textbf{Spam}|x)$

Multinomial.ipynb

File Edit View Insert Runtime Tools Help

+ Code + Text

[1]
```python
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report
```

[2]
```python
# Sample dataset
data = [
    ("Free money now!!!", "spam"),
    ("Call me when you can", "ham"),
    ("Win a free ticket to Bahamas!", "spam"),
    ("Important meeting scheduled at 3 PM", "ham"),
    ("Congratulations! You won a lottery!", "spam"),
    ("Can we reschedule our lunch?", "ham"),
]
```

```
[3] # Split data into text and labels
    texts, labels = zip(*data)
```

The * operator unpacks the list of tuples, so **zip()** can group the elements at the same positions in each tuple.

```
[4] texts
```

The **first elements** of all tuples are grouped into **one sequence**.

```
('Free money now!!!',
 'Call me when you can',
 'Win a free ticket to Bahamas!',
 'Important meeting scheduled at 3 PM',
 'Congratulations! You won a lottery!',
 'Can we reschedule our lunch?')
```

The **Second elements** of all tuples are grouped into one sequence

```
labels
```

```
('spam', 'ham', 'spam', 'ham', 'spam', 'ham')
```

```
[20] # Step 1: Convert text into numerical format using CountVectorizer
     vectorizer = CountVectorizer()
     x = vectorizer.fit_transform(texts)
```

CountVectorizer **will also** extract unique features from the texts.

CountVectorizer outputs a **sparse matrix**, which is efficient for storing text data with Numbers.

Multinomial.ipynb

File Edit View Insert Runtime Tools Help All changes saved

+ Code + Text

```python
# Convert to dense array
x_dense = x.toarray()
print(x_dense)
```

```
[[0 0 0 0 0 1 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0]
 [0 0 1 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 1 0 0 1]
 [0 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 1 0 0]
 [1 0 0 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 0 0 0 0 0 0]
 [0 0 0 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1]
 [0 0 0 1 0 0 0 0 1 0 0 0 0 1 0 1 0 0 0 1 0 0 0 0]]
```

Free Money Now

Call me when you can

```python
# Get feature names
print(vectorizer.get_feature_names_out())
```

```
['at' 'bahamas' 'call' 'can' 'congratulations' 'free' 'important'
 'lottery' 'lunch' 'me' 'meeting' 'money' 'now' 'our' 'pm' 'reschedule'
 'scheduled' 'ticket' 'to' 'we' 'when' 'win' 'won' 'you']
```

This will print a **2D array** where:

Rows correspond to the input texts.

Columns correspond to the unique words (features) in the vocabulary.

Values are the count of each word in the respective text.

+ Code + Text

[23]
```
# Step 2: Split data into training and testing sets
x_train, x_test, y_train, y_test = train_test_split(x,labels, test_size=0.3, random_state=42)
```

```
# Step 3: Train Multinomial Naive Bayes model
model = MultinomialNB()
model.fit(x_train, y_train)
```

```
MultinomialNB
MultinomialNB()
```

```
[27] # Step 4: Make predictions
     y_pred = model.predict(x_test)
```

vectorizer is the previously trained **CountVectorizer** object.

Ensures the **new text** uses the **same vocabulary (features)** **learned** during **training.**

```
[28] # Test new data
     new_texts = ["Win an iPhone now!", "Meeting at 5 PM"]
     new_X = vectorizer.transform(new_texts)
     predictions = model.predict(new_X)
     print("Predictions for new texts:", predictions)
```

```
Predictions for new texts: ['spam' 'ham']
```

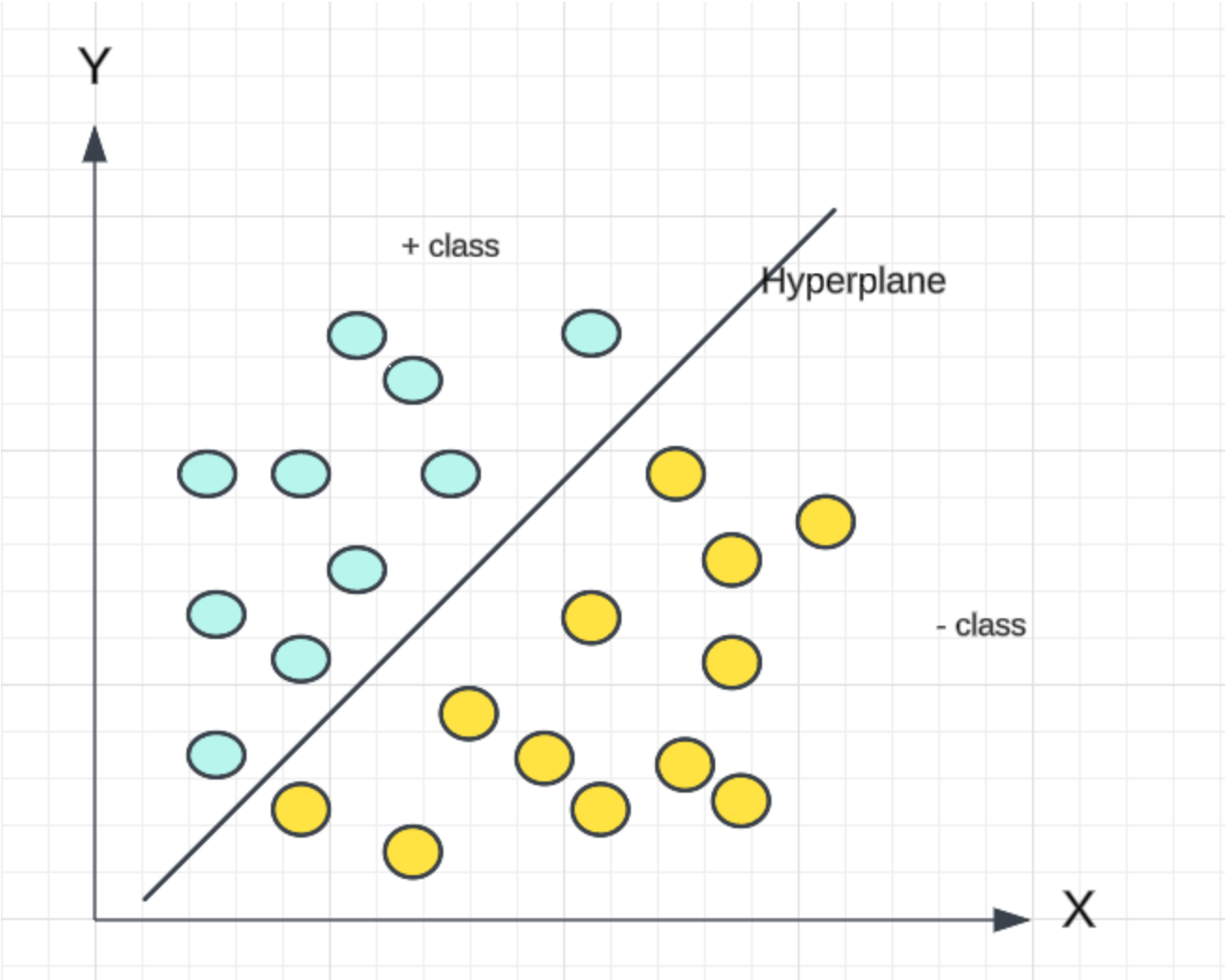
Predicted output

model.predict(new_X) uses **the trained Multinomial Naive Bayes model** to classify the **transformed new data (new_X)**.

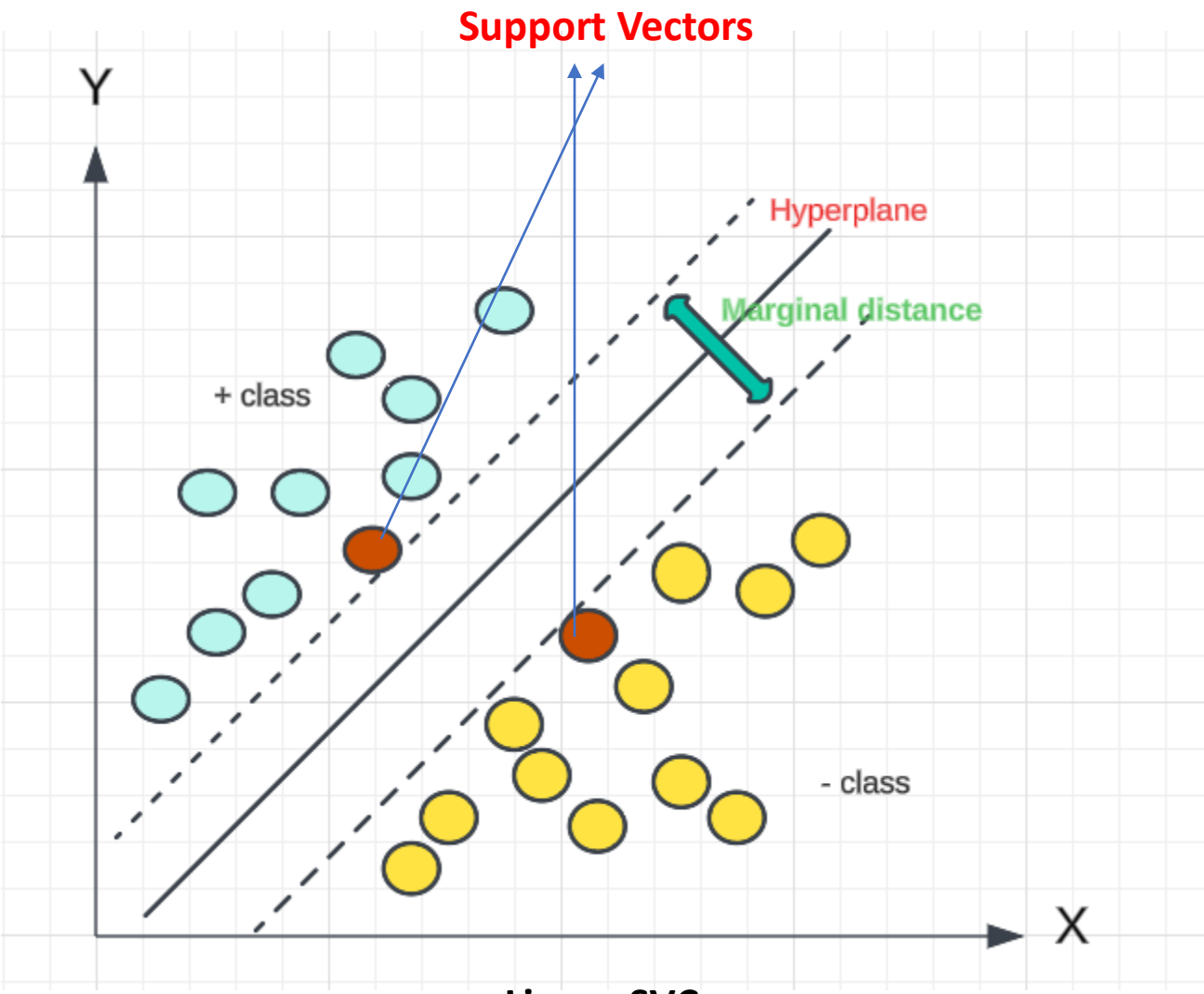
```
[29] # Convert to dense array
     x_newdense = new_X.toarray()
     print(x_newdense)
```

```
[[0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0]
 [1 0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 0 0 0]]
```

SUPPORT VECTOR MACHINE

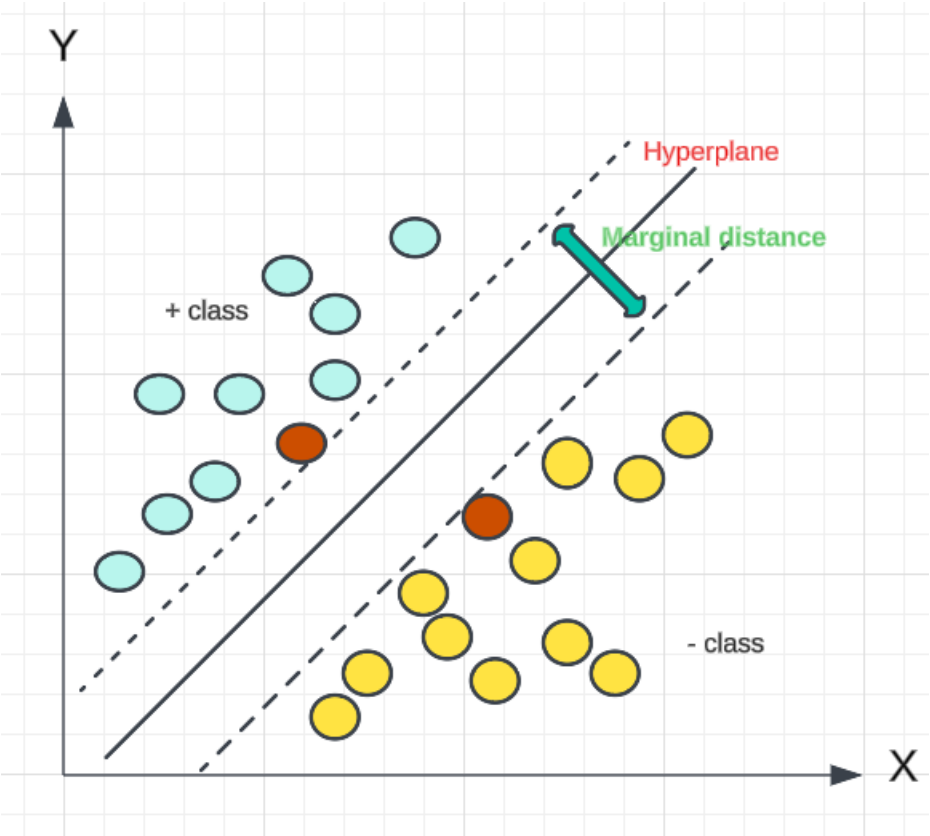


This kind of Binary **classification** we have seen in **Logistic regression**

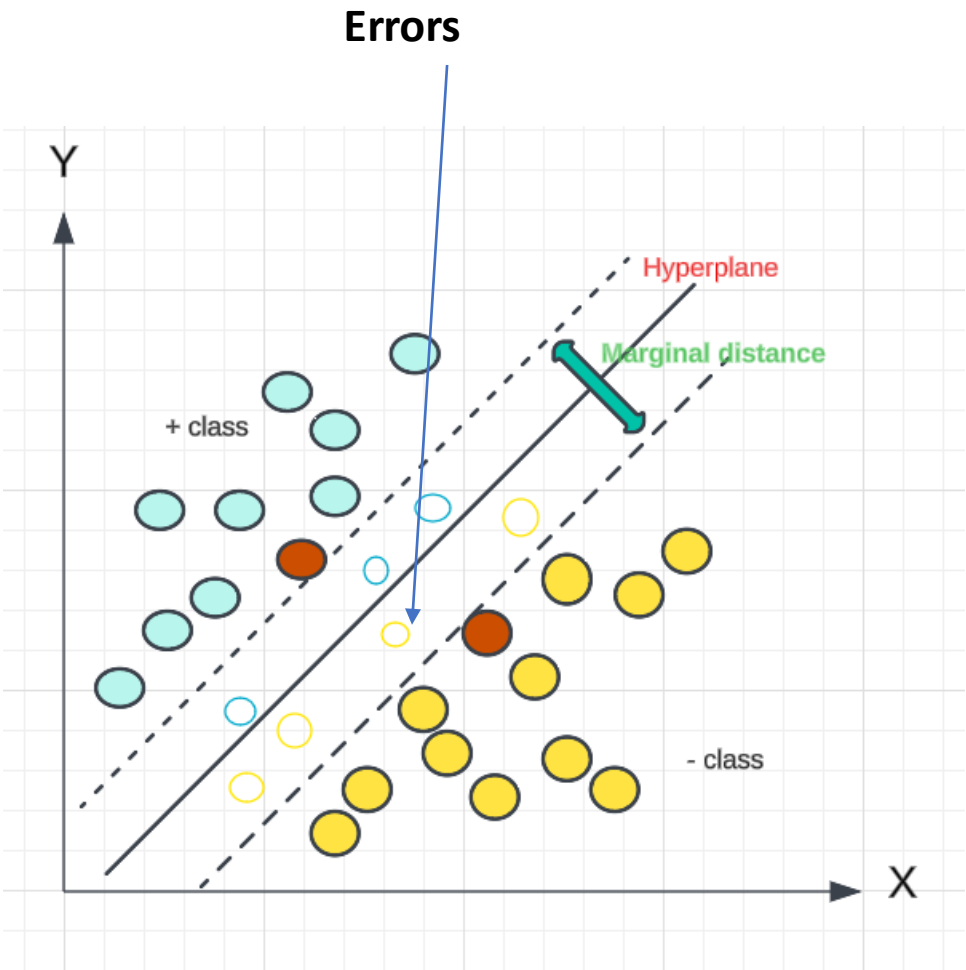


Linear SVC

- Apart from Creating **Hyperplane**
- **SVM** also creates two **Additional hyperplanes** in **parallel called Margins**
- This **Margin hyper planes should** have maximum distance **to linearly classify the data points.**
- This distance between two margins called **Marginal distance.**
- This **Hyperplanes** will **pass on through** the one or more of the **positive points** and **Negative points.**
- **Support vectors** is nothing but the **points** through which **marginal plane passes.**
- **Higher the Marginal distance more Generalize our Model is**



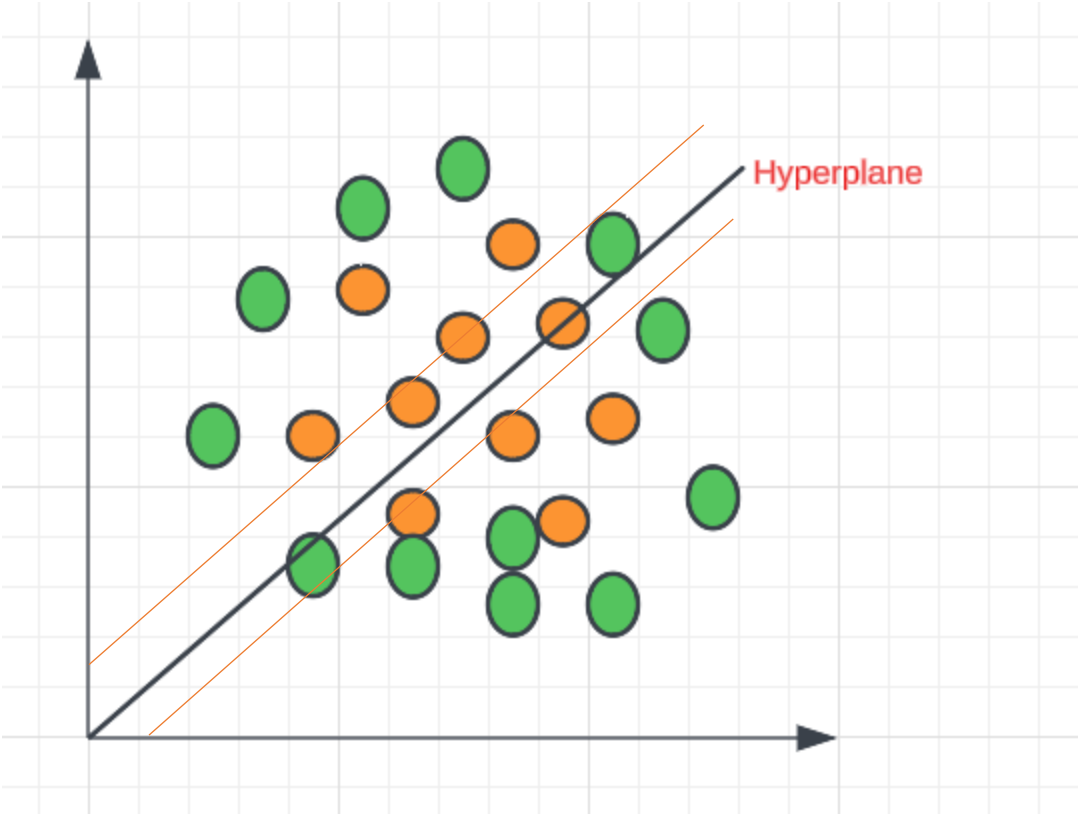
In this scenario the points are **clearly separated**. We don't have any **errors.** So this is called **Hard Margin**



In this case we have **some errors** so we will call it as **Soft Margin.**

In Real time scenario obviously, we will have some **overlap points**.

SVM Kernel

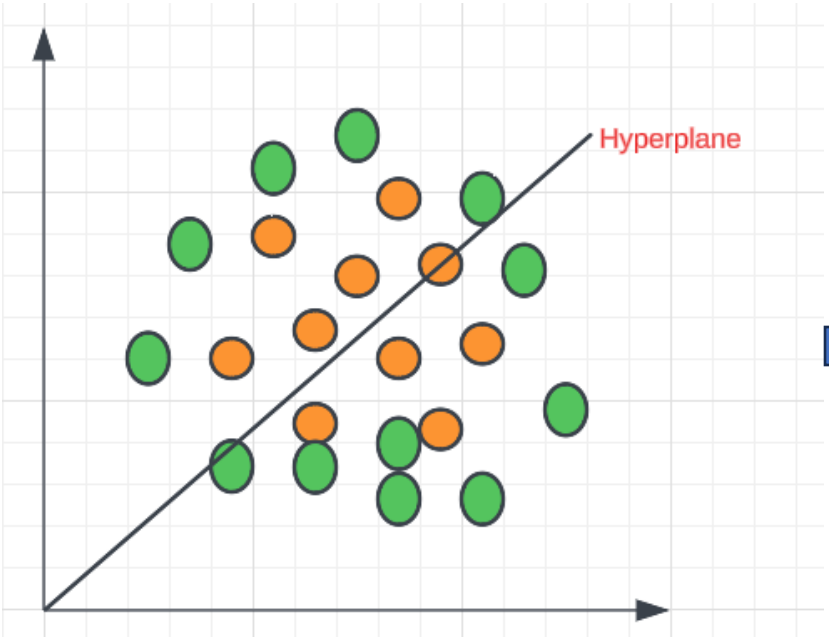


In **Support Vector Machines (SVM),** a **kernel** is a function that transforms data into a **higher-dimensional space**, enabling the **SVM** to **handle non-linear relationships** in the data.

- This is an example for **Non-Linear Separable** data points.
- Because both **positive and Negative** points are overlapped.
- Here **Linear SVC** will Not work.
- If we use Linear SVC here our **Accuracy will be low** and **errors will be high**.

SVM Kernel

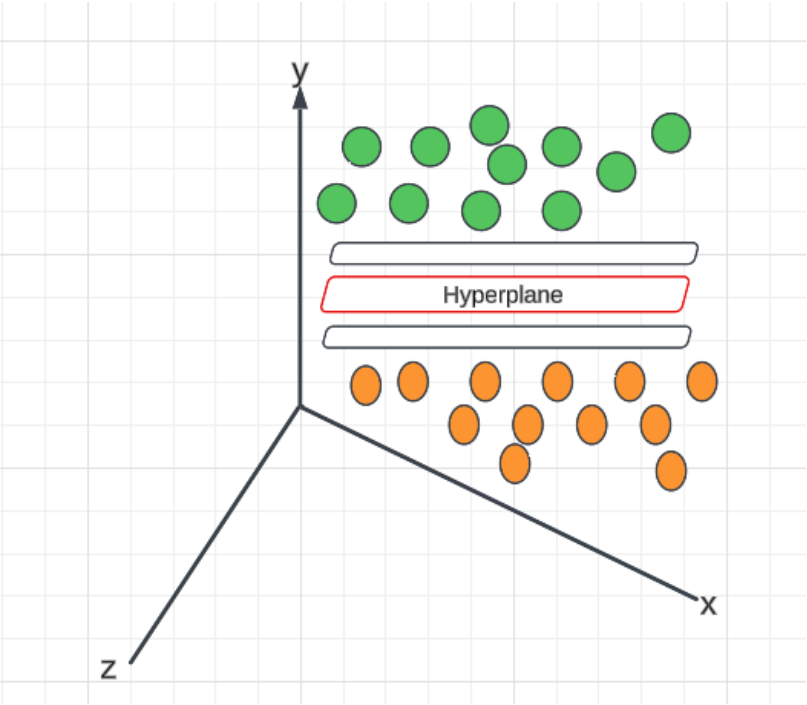
2D



SVM Kernel

- **Polynomial**
- **Radial Basis Function (RBF) Kernel**
- **Sigmoid Kernel**

3D



With the help of **linear SVC** we can create **3D Hyper plane**

SVC_Implementation.ipynb

File Edit View Insert Runtime Tools Help Last saved at 8:25 PM

+ Code + Text

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

[2]
```python
#lets create a synthetic data point
from sklearn.datasets import make_classification
```

```
[49] x,y=make_classification(n_samples=1000,n_features=2,n_classes=2,n_clusters_per_class=1,n_redundant=0)
```

```
[50] x
```

```
array([[-1.33304895, -0.85363495],
       [ 0.20585848, -1.2908314 ],
       [ 1.23475894,  0.07443327],
       ...,
       [ 1.32001971,  0.30194453],
       [ 1.34678856,  1.04633958],
       [ 0.52566049,  1.22738382]])
```

```
[51] y
```

```
array([1, 1, 0, 1, 1, 1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 0, 1, 1, 0, 0,
       0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0,
       0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1,
       1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0,
       0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1,
       0, 1, 1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1,
       1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1,
       0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0,
       1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0,
       1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0,
       0, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 1, 1,
       0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0,
       1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1, 0, 0,
       1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0,
       1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 0,
       0, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0, 0, 1, 0, 1,
```

SVC_Implementation.ipynb

File Edit View Insert Runtime Tools Help All changes saved

+ Code + Text

```
pd.DataFrame(x)
```

	0	1
0	-1.333049	-0.853635
1	0.205858	-1.290831
2	1.234759	0.074433
3	-1.416418	-0.612760
4	-0.014548	-1.495005
...	...	...
995	0.569460	1.539963
996	1.515368	0.336041
997	1.320020	0.301945
998	1.346789	1.046340
999	0.525660	1.227384

1000 rows × 2 columns

x-axis values for the scatter plot, which correspond to the values in the **column "0"** of the **DataFrame df.**

y-axis values for the scatter plot, which correspond to the values in the **column "1"** of the **DataFrame df.**

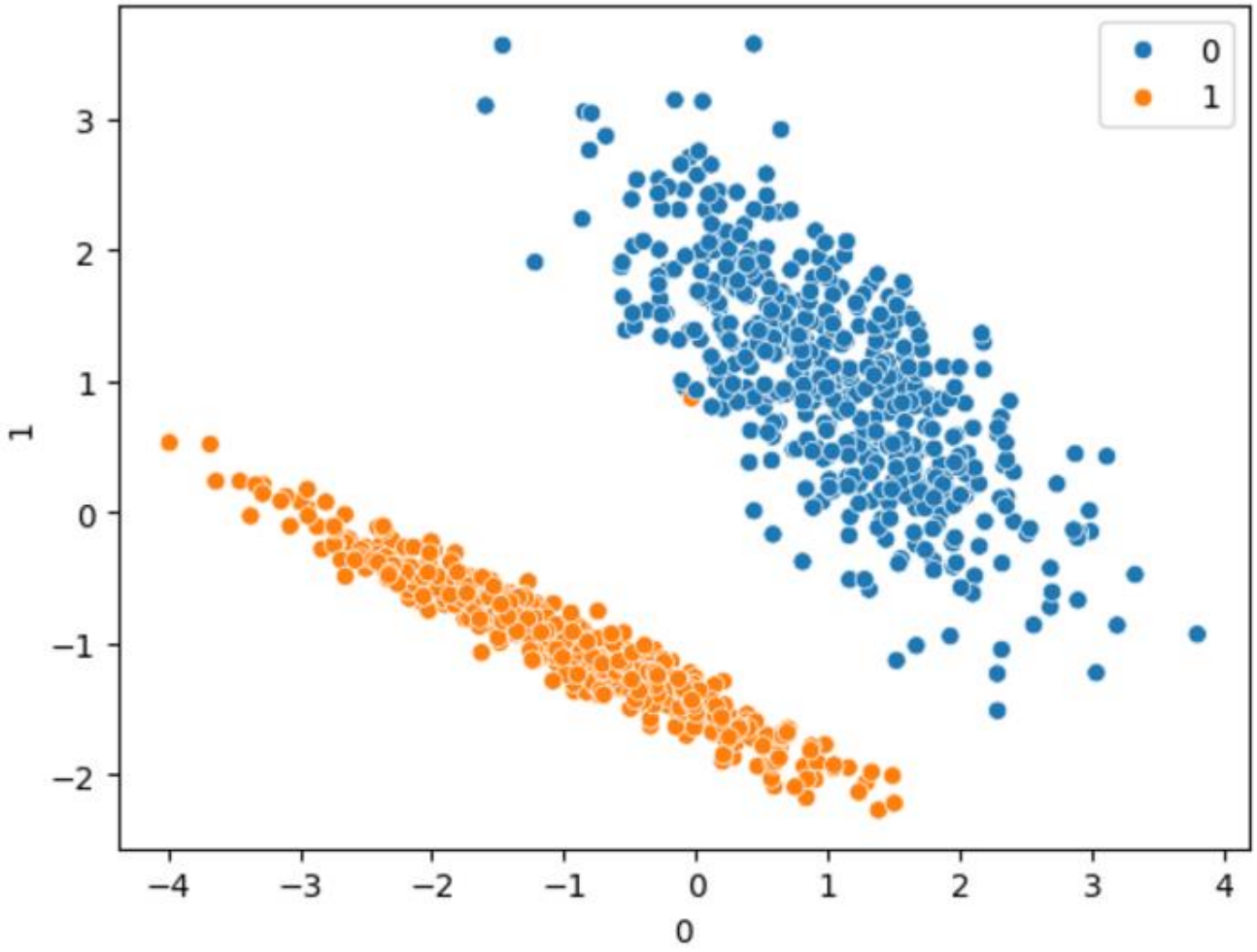
points belonging to **class 0** will have one **color**, and points in **class 1** will have another **color**.

```
[53] # Convert x to a DataFrame for column referencing
     df = pd.DataFrame(x, columns=["0", "1"])
```

```
[54] # Plot with hue based on y
     sns.scatterplot(x=df["0"], y=df["1"], hue=y)
```

+ Code + Text

```
[54] # Plot with hue based on y
     sns.scatterplot(x=df["0"], y=df["1"], hue=y)
     plt.show()
```



Now here you can definitely see that, uh, **my data points are very clearly separable.**

And obviously I can actually create a **best fit line** with the help of **SVC.**

And for this, the **SVC** that we will be using, it is called as **linear SVC**.

SVC_Implementation.ipynb

File Edit View Insert Runtime Tools Help All changes saved

+ Code + Text

[55]
```
#Train test split
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
```

[57]
```
from sklearn.svm import SVC
```

[58]
```
# Create an SVM classifier
model = SVC(kernel="linear")
```

```
model.fit(x_train,y_train)
```

```
SVC
SVC(kernel='linear')
```

SVC_Implementation.ipynb

File Edit View Insert Runtime Tools Help Saving...

+ Code + Text

[60] Suggested code may be subject to a license | TABEYWICKRAMA/ML-series-end-to-end

```
#prediction
y_pred = model.predict(x_test)
```

[61]

```
#perfomance metrics
from sklearn.metrics import accuracy_score,classification_report,confusion_matrix
```

```
print(accuracy_score(y_test,y_pred))
print(classification_report(y_test,y_pred))
print(confusion_matrix(y_test,y_pred))
```

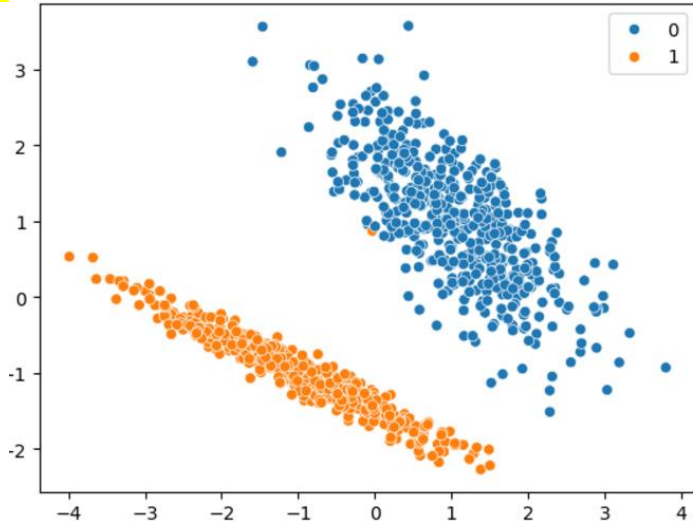
```
1.0
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       100
           1       1.00      1.00      1.00       100

    accuracy                           1.00       200
   macro avg       1.00      1.00      1.00       200
weighted avg       1.00      1.00      1.00       200

[[100   0]
 [  0 100]]
```

100% Accuracy because our dataset is linearly separable



```
[64] x,y=make_classification(n_samples=1000,n_features=2,n_classes=2,n_clusters_per_class=2,n_redundant=0)
```

```
[65] x
```

```
array([[ 1.27324438, -0.1248718 ],
       [-1.22076874, -0.04004501],
       [-1.51214383,  1.07648029],
       ...,
       [-1.85592072,  0.16752663],
       [ 2.14176148,  0.0851422 ],
       [ 0.95524249,  0.8922086 ]])
```

In this parameter I changed the cluster group to **2,** previously we gave it as **1**

Run this code again

```
y
```

```
array([1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 1,
       1, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 1,
       1, 0, 1, 1, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 1,
       0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0,
       1, 1, 0, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1,
       0, 0, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1, 0, 1,
       0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0, 1,
       0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1,
       0, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1,
```

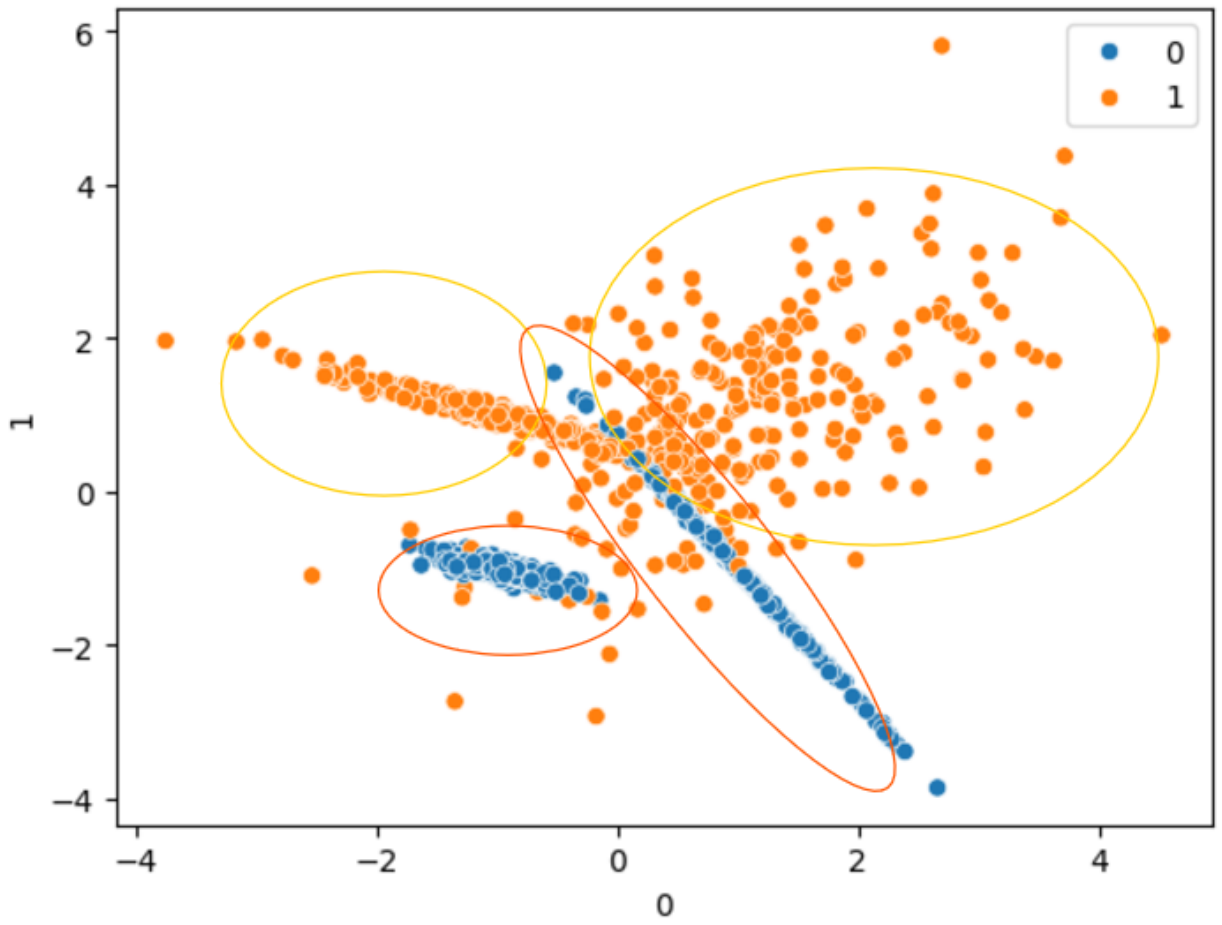
n_clusters_per_class=1, each class **class 0** and **class1** will form it's own single cluster.

n_clusters_per_class > 1 each class **class 0** and **class1** will form more than one cluster.

+ Code + Text

```
[73] # Convert x to a DataFrame for column referencing
     df = pd.DataFrame(x, columns=["0", "1"])
```

```
# Plot with hue based on y
sns.scatterplot(x=df["0"], y=df["1"], hue=y)
plt.show()
```



This time we can see **more than 2 clusters** of each class and **Non linear separable data** i.e. data overlapping is there

```
[75] #Train test split
     from sklearn.model_selection import train_test_split
     x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
```

```
[76] from sklearn.svm import SVC
```

```
[77] # Create an SVM classifier
     model = SVC(kernel="linear")
```

```
[78] model.fit(x_train,y_train)
```

```
SVC
SVC(kernel='linear')
```

```
[79] Suggested code may be subject to a license | TABEYWICKRAMA/ML-series-end-to-end
     #prediction
     y pred = model.predict(x test)
```

Train and Run the Model again for new dataset.

```
[14] #perfomance metrics
     from sklearn.metrics import accuracy_score,classification_report,confusion_matrix
```

```
print(accuracy_score(y_test,y_pred))
print(classification_report(y_test,y_pred))
print(confusion_matrix(y_test,y_pred))
```

```
0.845
              precision    recall  f1-score   support

           0       0.89      0.80      0.84       105
           1       0.80      0.89      0.85        95

    accuracy                           0.84       200
   macro avg       0.85      0.85      0.84       200
weighted avg       0.85      0.84      0.84       200

[[84 21]
 [10 85]]
```

This time you can see
Accuracy is reduced to 84%

Kernel Implementation: RBF (Radial Basis Function kernel), is a popular kernel used in Support Vector Machines (SVMs) for non-linear classification tasks.

```
[82] #Rbf kernel
     model2=SVC(kernel="rbf")
```

```
[83] model2.fit(x_train,y_train)
```

```
SVC
SVC()
```

```
y_pred2=model2.predict(x_test)
```

```python
print(accuracy_score(y_test,y_pred2))
print(classification_report(y_test,y_pred2))
print(confusion_matrix(y_test,y_pred2))
```

```
0.885
              precision    recall  f1-score   support

           0       0.95      0.83      0.88       105
           1       0.83      0.95      0.89        95

    accuracy                           0.89       200
   macro avg       0.89      0.89      0.88       200
weighted avg       0.89      0.89      0.88       200

[[87 18]
 [ 5 90]]
```

When Using **Rbf** , we can see a slight improvement in **accuracy 89%**

Hyper Parameter Tuning

```
[33] from sklearn.model_selection import GridSearchCV
```

```
[34] #Defining paremeter range
     param_grid = {'C': [0.1, 1, 10, 100],
                   'gamma': [1, 0.1, 0.01, 0.001],
                   'kernel': ['rbf']}
```

```
# GridSearchCV for hyperparameter tuning
grid = GridSearchCV(SVC(), param_grid=param_grid,cv=5, verbose=3)
grid.fit(x_train, y_train)
```

verbose=3 means **detailed logging** of the progress during computation.

- verbose=0: No output.
- verbose=1: Basic progress updates.
- verbose=2: More detailed updates about folds
- verbose=3: The most detailed information about parameter combinations and folds

```
Fitting 5 folds for each of 16 candidates, totalling 80 fits
[CV 1/5] END .......C=0.1, gamma=1, kernel=rbf;, score=0.919 total time=   0.0s
[CV 2/5] END .......C=0.1, gamma=1, kernel=rbf;, score=0.844 total time=   0.0s
[CV 3/5] END .......C=0.1, gamma=1, kernel=rbf;, score=0.887 total time=   0.0s
[CV 4/5] END .......C=0.1, gamma=1, kernel=rbf;, score=0.906 total time=   0.0s
[CV 5/5] END .......C=0.1, gamma=1, kernel=rbf;, score=0.875 total time=   0.0s
[CV 1/5] END .....C=0.1, gamma=0.1, kernel=rbf;, score=0.887 total time=   0.0s
[CV 2/5] END .....C=0.1, gamma=0.1, kernel=rbf;, score=0.812 total time=   0.0s
[CV 3/5] END .....C=0.1, gamma=0.1, kernel=rbf;, score=0.887 total time=   0.0s
[CV 4/5] END .....C=0.1, gamma=0.1, kernel=rbf;, score=0.900 total time=   0.0s
[CV 5/5] END .....C=0.1, gamma=0.1, kernel=rbf;, score=0.850 total time=   0.0s
[CV 1/5] END ....C=0.1, gamma=0.01, kernel=rbf;, score=0.875 total time=   0.0s
[CV 2/5] END ....C=0.1, gamma=0.01, kernel=rbf;, score=0.800 total time=   0.0s
[CV 3/5] END ....C=0.1, gamma=0.01, kernel=rbf;, score=0.856 total time=   0.0s
[CV 4/5] END ....C=0.1, gamma=0.01, kernel=rbf;, score=0.875 total time=   0.0s
```

C:

- Regularization parameter for SVM.
- To achieve a **low error on the training data** and **maintaining a large margin (generalization).**
- Larger values of **C** make the model **stricter on margin violations.**

Gamma:

- Kernel coefficient for the **RBF kernel** (controls influence of individual training points).
- It controls the **shape of the decision boundary** that the **SVM tries to create to separate the classes.**
- High gamma Can lead to overfitting
- Low gamma can lead to underfitting

[47]

```
GridSearchCV
best_estimator_: SVC
SVC
```

[48]

```
# Best parameters
print("Best parameters found:", grid.best_params_)
```

```
Best parameters found: {'C': 10, 'gamma': 1, 'kernel': 'rbf'}
```

[49]

```
#prediction
y_pred3=grid.predict(x_test)
```

```
print(accuracy_score(y_test,y_pred3))
print(classification_report(y_test,y_pred3))
print(confusion_matrix(y_test,y_pred3))
```

```
0.89
              precision    recall  f1-score   support

           0       0.95      0.84      0.89       105
           1       0.84      0.95      0.89        95

    accuracy                           0.89       200
   macro avg       0.89      0.89      0.89       200
weighted avg       0.90      0.89      0.89       200

[[88 17]
 [ 5 90]]
```

Very slight improvement in Accuracy %